A PRIMER OF MAXIMUM LIKELIHOOD PROGRAMMING IN R

Marco R. Steenbergen

2025-10-28

Abstract

R is an outstanding platform for maximum likelihood estimation. These notes describe the `maxLik` package, a wrapper that gives access to the most important hill-climbing algorithms and provides a convenient way of displaying results. The notes also include a brief discussion of Wald and profile confidence intervals, likelihood ratio tests, and simulation.

Versions and Notation

In these notes, we rely on the following versions of R and `maxLik`.

```
[1] "R version 4.5.0 (2025-04-11)"
```

```
[1] "1.5-2.1"
```

Probability mass functions are denoted as $P(.)$, whereas probability density functions are indicated with $p(.)$. The log-likelihood for a single sampling unit is indicated by ℓ_i. The log-likelihood over the data is given by $\ell = \sum_{i=1}^{n} \ell_i$, where n denotes the sample size.

Introduction

By now, R is one of the most widely used programming languages in the social and behavioral sciences. Open source and available for all major operating systems, R provides a versatile and precise environment for statistical computing. This includes maximum likelihood estimation.

R contains numerous routines for optimizing home-made likelihood functions. The `maxLik` package provides an easy method of applying those routines (Henningsen and Toomet 2011). Among the implemented hill-climbing algorithms are Newton-Raphson, BFGS (Broyden 1970; Fletcher 1970; Goldfarb 1970; Shanno 1970), and BHHH (Berndt et al. 1974).[1] The package is capable of numeric differentiation, a feature we shall use in most of our examples. It produces a uniform-looking output, regardless of the optimizer that was used.

Syntactic Structure

Evaluating likelihood functions entails a two-step process. First, one declares the log-likelihood function. Next, one optimizes that function. The log-likelihood function and optimization command can be typed interactively into the command window or they can be contained in a `.R` syntax file. I suggest saving log-likelihood functions into a file, especially if you plan on using them frequently. They can then be accessed using the `source` function.

Declaring the Log-Likelihood

The log-likelihood function is a normal function in R. It takes two arguments: (1) a parameter or vector of parameters and (2) a data object. The latter can be a data frame, matrix, or vector. It designates the name the log-likelihood uses internally to reference the data. In the optimization, this can be set equal to an object with a different name. Sometimes, it can be helpful to break `df` up into multiple objects.

The generic syntax takes the following form

```
name <- function(pars, df) {
  declarations
  lnl <- expression
  return(lnl)
}
```

Here, `pars` is the name of the parameter (vector) and `df` is the name of the data frame or matrix. Under `declarations`, one can partial out the parameter vector and generate temporary variables that can be helpful for specifying the log-likelihood. The log-likelihood itself is specified as the object `lnl`. The function returns this object for access by `maxLik`. The precise specification of the log-likelihood depends on the optimizer: it is different for the BHHH-algorithm than for other optimizers.

[1] For a general discussion of hill-climbing algorithms see Thisted (1988).

Single Parameter

We start with some simple examples, where`declarations` can be left empty.

> **Example 1:** Consider the Poisson probability mass function with unit log-likelihood functions of
>
> $$\ell_i = \ln P(y_i|\mu) = -\mu + y_i \ln \mu - \ln(y_i!)$$
>
> We program this using

```
lnl_poisson1 <- function(mu, y) {
  lnl <- -mu + y * log(mu) - lfactorial(y)
  return(lnl)
}
```

> The example specifies `y` as the placeholder for a data vector/dataframe and has only the parameter μ as an argument. The function `lfactorial(y)` returns the natural logarithm of the factorial of y. We might have also omitted this term because it does not contain the parameter and, as such, is irrelevant for the optimization. The function returns the log-likelihood for a single observation: ℓ_i.

This declaration of the log-likelihood suitable for the BHHH-algorithm. It does not work for other algorithms, however, because they expect ℓ in lieu of ℓ_i. We address this issue in the next example, where I have also omitted the reference to the log-factorial.

> **Example 2:** For the Poisson distribution, the full log-likelihood is (up to an additive constant) given by
>
> $$\ell = -n\mu + \left(\sum_{i=1}^{n} y_i\right) \ln \mu$$
>
> Assuming the data are stored in the vector `y`, we can program the log-likelihood as follows.

```
lnl_poisson2 <- function(mu, y) {
  n <- length(y)
  lnl <- -n * mu + sum(y) * log(mu)
  return(lnl)
}
```

Note that if `y` is a data frame, `length(y)` should be replaced with `nrow(y)`.

In the examples so far, I used the formula of the probability mass function to generate the log-likelihood. However, R has a built-in function for the Poisson distribution that we can also use the program the log-likelihood. This brings us to yet another way in which we can specify the log-likelihood.

Example 3: We can use `dpois` to specify the full log-likelihood.

```
lnl_poisson3 <- function(mu, y) {
  lnl <- log(dpois(y, lambda = mu))
  return(sum(lnl))
}
```

I assign the natural logarithm of the probability mass function to `lnl`. This is equivalent to ℓ_i. The summation (needed for obtaining ℓ) is performed within the `return` of the function.

Multiple Parameters

When the log-likelihood contains multiple parameters, then `pars` is a vector. We typically need to partition this vector into subsets of parameters. For instance, the normal distribution may be written as $y \sim N(\theta)$. Here, θ is a vector consisting of two parameters: (1) μ and (2) σ. We can define those as elements of the `pars` vector when specifying the log-likelihood.

Example 4: For the normal distribution with parameters μ and σ, the kernel of the log-likelihood is given by

$$\ell = -n \ln \sigma - .5 \sum_{i=1}^{n} \left\{ \frac{(y_i - \mu)^2}{\sigma^2} \right\}$$

When we define $z = (y - \mu)/\sigma$, then the log-likelihood kernel may also be written as

$$\ell = -n \ln \sigma - .5 \sum_{i=1}^{n} z_i^2$$

We can now define the normal log-likelihood in R using

```
lnl_normal <- function(pars, y) {
  mu <- pars[1]
  sigma <- pars[2]
  n <- length(y)
  z <- (y - mu)/sigma
  lnl <- -n * log(sigma) - .5 * sum(z^2)
  return(lnl)
}
```

Line 2 declares the first parameter in `pars` to be equal to μ, whereas line 3 sets the second parameter equal to σ. In line 4, the variables z are created. Line 5 declares the log-likelihood function, whereas line 6 returns it.

Optimization

Once the log-likelihood has been declared, `maxLik` is invoked to perform the optimization of this function. The generic syntax is:

```
library(maxLik)
object <- maxLik(loglikelihood,
                 grad =,
                 hess =,
                 method = c("BFGS", "BFGSR", "BHHH",
                            "CG", "NM", "NR", "SANN"),
                 start =,
                 data)
```

Log-Likelihood, Gradient, and Hessian

The argument `loglikelihood` is mandatory and is the function that we specified earlier. When using numerical derivatives, the arguments `grad` and `hess` (for gradient and hessian) can both be set equal to `NULL`. Since this the default, we could also skip adding those arguments altogether when using numerical derivatives. Later, we shall explore examples where we specify the gradient and hessian as functions.

Method

The `method` argument declares the optimizer that should be used. The current choices are:

- `BFGS` for Broyden-Fletcher-Goldfarb-Shanno (Broyden 1970; Fletcher 1970; Goldfarb 1970; Shanno 1970).
- `BFGSR` for the R implementation of the Broyden-Fletcher-Goldfarb Shanno algorithm.
- `BHHH` for Berndt-Hall-Hall-Hausman (Berndt et al. 1974).
- `CG` for conjugate gradients (Shewchuk 1994).
- `NM` for Nelder-Mead (Nelder and Mead 1965).
- `NR` for Newton-Raphson, which builds on the work of Isaac Newton and Josephh Raphson from the 17th century.
- `SANN` for simulated annealing (Kirkpatrick, Gelatt, and Vecchi 1983).

Starting Values

The argument `start` declares the starting value(s). When a single parameter is estimated, the declaration is simply

```
start = #
```

where `#` is a number. This number should be chosen according to the range restrictions on the parameter space. For instance, in the Poisson distribution, μ has to be strictly positive. Specifying a starting value that is outside this range will cause a warning such as `NaNs produced`.

If the log-likelihood contains multiple parameters, then a vector of starting values should be declared, again in accordance with the range restrictions:

```
start = c(#1, #2, ...)
```

Here `#1`, `#2`, etc. are successive starting values that should be specified in the same order as the parameters.

It is possible to use previous estimation results as starting values, as some of the examples will show.

Data

The object `data` specifies the data frame or a matrix/vector containing the data. This is passed into the log-likelihood function. It is important to ensure that the data are consistent with the support of the probability density/mass function underlying the log-likelihood. If this is not the case, `maxLik` will generate an error.

Output

`maxLik` yields a number of values automatically:

- `AIC` contains information about Akaike's information criterion (Akaike 1973).
- `coeff` contains the parameter estimates.
- `logLik` contains the log-likelihood value upon convergence of the optimizer.
- `nIter` shows the number of iterations that an optimizer need to reach convergence.
- `nParam` contains the number of parameters.
- `stdEr` contains the standard errors of the estimators.
- `summary` produces a summary table of results (as shown below).
- `vcov` contains the estimated variance-covariance matrix of the estimators.

Examples

Example 5: We start by generating 1000 observations from a $P(2)$-distribution.

```
set.seed(10)
y_poisson <- rpois(1000, lambda = 2)
```

For this example, I use the BHHH algorithm and apply it to `lnl_poisson1`:

```
poisson_fit1 <- maxLik(lnl_poisson1,
                       method = "BHHH",
                       start = 1,
                       y = y_poisson)
summary(poisson_fit1)
```

```
--------------------------------------------
Maximum Likelihood estimation
BHHH maximisation, 7 iterations
Return code 8: successive function values within relative tolerance limit (reltol)
Log-Likelihood: -1748.701
```

```
1  free parameters
Estimates:
     Estimate Std. error t value Pr(> t)
[1,]  2.04998    0.04316   47.49  <2e-16 ***
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
--------------------------------------------
```

The anatomy of the output is as follows:

- Line 2 shows that we used the BHHH algorithm and that it took 7 iterations to converge.
- The return code on line 3 shows the algorithm converged. The return codes vary across optimizers. However, the message that follows it contains sufficient information to decipher whether convergence succeeded. Here, the message says that successive function values (of the log-likelihood) were within the relative tolerance limit. This means that the improvements in the log-likelihood were so small that further iterations were deemed unnecessary.[2]
- Line 4 shows the value of the log-likelihood upon convergence.
- Line 5 indicates there was one free parameter that was estimated, to wit μ.
- The remainder of the output shows the maximum likelihood estimate (2.05), the estimated standard error (0.043), the test statistic for the null hypothesis that $\mu = 0$, and the p-value associated with that statistic.[3]

The following example shows how we can access various values in the `poisson_fit1` object.

Example 6:

```
# Log-likelihood and degrees of freedom
logLik(poisson_fit1)
```

```
[1] -1748.701
attr(,"df")
[1] 1
```

[2]The convergence codes for the specific optimization algorithms are as follows: 0 for BFGS, CG, NM, and SANN; 2 for BFGSR, BHHH, and NR.

[3]The p-value is computed by referring the test statistic to the standard normal distribution. This reflects the asymptotic property that maximum likelihood estimators are normally distributed.

```
# Estimates
coef(poisson_fit1)
```

```
[1] 2.049985
```

```
# Standard errors
stdEr(poisson_fit1)
```

```
[1] 0.04316475
```

```
# Alternatively, variance-covariance matrix
vcov(poisson_fit1)
```

```
            [,1]
[1,] 0.001863195
```

```
# Number of estimated parameters
nParam(poisson_fit1, free = TRUE)
```

```
[1] 1
```

We now look at optimizing the Poisson log-likelihood using the Newton-Raphson algorithm. This requires the use of `lnl_poisson2` (or `lnl_poisson3`).

Example 7:

```
poisson_fit2 <- maxLik(lnl_poisson2,
                       method = "NR",
                       start = 1,
                       y = y_poisson)
summary(poisson_fit2)
```

```
--------------------------------------------
Maximum Likelihood estimation
Newton-Raphson maximisation, 5 iterations
Return code 8: successive function values within relative tolerance limit (reltol)
Log-Likelihood: -578.4284
1  free parameters
Estimates:
     Estimate Std. error t value Pr(> t)
[1,]  2.05000    0.04526   45.29  <2e-16 ***
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
--------------------------------------------
```

Comment: You will notice that the parameter estimate is virtually indistinguishable from Example 5. The same is not true for the estimated standard error, however. This is due to differences in the way the variance-covariance matrix of the estimators is computed. The BHHH algorithm uses the outer-product of gradients, whereas the Newton-Raphson algorithm uses the observed Fisher information. Asymptotically, these differences vanish.

I conclude this section by describing an example with multiple parameters, to wit the normal distribution. Again, we start by simulating some data, in this case from $N(5, 10)$. Then, we optimize `lnl_normal` using the Newton-Raphson algorithm. The example shows how starting values can be specified as an object that is then called in `maxLik`.

Example 8:

```
set.seed(20)
y_normal <- rnorm(500, mean = 5, sd = 10)
start <- c(0, 1) # Starting values of a standard normal distribution
normal_fit1 <- maxLik(lnl_normal,
                      start = start,
                      method = "NR",
                      y = y_normal)
summary(normal_fit1)
```

```
--------------------------------------------
Maximum Likelihood estimation
Newton-Raphson maximisation, 13 iterations
Return code 8: successive function values within relative tolerance limit (reltol)
Log-Likelihood: -1399.388
2  free parameters
Estimates:
     Estimate Std. error t value Pr(> t)
[1,]   4.9810     0.4283   11.63  <2e-16 ***
[2,]   9.9620     0.3238   30.77  <2e-16 ***
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
--------------------------------------------
```

Fine-Tuning

Analytic Derivatives

In some cases, numeric derivatives can be unreliable or time consuming to compute and it can pay off to provide analytic derivatives instead. One could opt to provide the gradient, the hessian, or both. Example 9 illustrates this for the normal distribution.

Example 9: Imagine we want to use the Newton-Raphson algorithm, which depends on gradient and hessian information. We decide to provide an explicit formula for gradient and hessian. Example 4 showed that the log-likelihood is $\ell = -n \ln \sigma - .5 \sum_{i=1}^{n} z_i^2$. The code for this is also provided in that example.

The gradient vector is

$$\nabla = \begin{bmatrix} \frac{\partial \ell}{\partial \mu} \\ \frac{\partial \ell}{\partial \sigma} \end{bmatrix} = \begin{bmatrix} \frac{1}{\sigma} \sum_{i=1}^{n} z_i \\ \frac{1}{\sigma} \left(\sum_{i=1}^{n} z_i^2 - n \right) \end{bmatrix}$$

We can code this as follows:

```
grad_normal <- function(pars, y) {
  mu <- pars[1]
  sigma <- pars[2]
  n <- length(y)
  z <- (y - mu)/sigma
  grad.vals <- numeric(2)
  grad.vals[1] <- (1/sigma) * sum(z)
  grad.vals[2] <- (1/sigma) *(sum(z^2) - n)
  return(grad.vals)
}
```

The hessian is given by

$$H = \begin{bmatrix} \frac{\partial^2 \ell}{\partial \mu \partial \mu} & \frac{\partial^2 \ell}{\partial \mu \partial \sigma} \\ \frac{\partial^2 \ell}{\partial \sigma \partial \mu} & \frac{\partial^2 \ell}{\partial \sigma \partial \sigma} \end{bmatrix} = \begin{bmatrix} -\frac{n}{\sigma^2} & -\frac{2}{\sigma^2} \sum_{i=1}^{n} z_i \\ -\frac{2}{\sigma^2} \sum_{i=1}^{n} z_i & -\frac{1}{\sigma^2} \left(3 \sum_{i=1}^{n} z_i^2 - n \right) \end{bmatrix}$$

The code for this function is as follows:

```r
hess_normal <- function(pars, y) {
  mu <- pars[1]
  sigma <- pars[2]
  n <- length(y)
  z <- (y - mu)/sigma
  hess.vals <- matrix(0, nrow = 2, ncol = 2)
  hess.vals[1,1] <- -n/sigma^2
  hess.vals[1,2] <- -(2/sigma^2) * sum(z)
  hess.vals[2,1] <- -(2/sigma^2) * sum(z)
  hess.vals[2,2] <- -(1/sigma^2) *(3 * sum(z^2) - n)
  return(hess.vals)
}
```

We apply this to the normal data we generated earlier.

```r
normal_fit2 <- maxLik(lnl_normal,
                      grad = grad_normal,
                      hess = hess_normal,
                      method = "NR",
                      start = c(0,1),
                      y = y_normal)
summary(normal_fit2)
```

```
--------------------------------------------
Maximum Likelihood estimation
Newton-Raphson maximisation, 13 iterations
Return code 1: gradient close to zero (gradtol)
Log-Likelihood: -1399.388
2  free parameters
Estimates:
     Estimate Std. error t value Pr(> t)
[1,]   4.9809     0.4455   11.18  <2e-16 ***
[2,]   9.9620     0.3150   31.62  <2e-16 ***
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
--------------------------------------------
```

Constrained Optimization

It is easy to impose linear constraints in the estimation by using the `fixed` option in the `maxLik` function. This allows us to specify which parameters should be estimated and which one should be fixed to their starting value. The following is an example of how to do this.

Example 10: For the data in Example 9, let us hypothesize H_0 : $\sigma = 1$. Reverting back to numerical derivatives, we can build this into the estimation in the following manner.

```
c_normal_fit <- maxLik(lnl_normal,
                       y = y_normal,
                       method = "NR",
                       start = c(0,1),
                       fixed = c(FALSE,TRUE))
summary(c_normal_fit)
```

```
--------------------------------------------
Maximum Likelihood estimation
Newton-Raphson maximisation, 4 iterations
Return code 8: successive function values within relative tolerance limit (reltol)
Log-Likelihood: -24810.29
1  free parameters
Estimates:
     Estimate Std. error t value Pr(> t)
[1,]  4.98091    0.04529     110  <2e-16 ***
[2,]  1.00000    0.00000      NA      NA
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
--------------------------------------------
```

The fixed command is set to `FALSE` for the first parameter, μ, which we seek to estimate. It is said to `TRUE` for σ, the parameter we do not want to estimate. For this parameter, we set the value to 1. This is the value that is provided for σ under the starting values. We see that the output shows an estimate of 1.000 for σ, with a standard error of 0. That standard error reflects the fact there is no sampling variability because nothing was estimated.

Re-Parameterization

Optimization works best when the parameter space is unconstrained, i.e., the parameter can take any real value. In this case, there is no risk that the optimizer becomes stuck at the boundary of parameter space. If a parameter, θ, does not already map onto $\mathbb{R}$, it can be useful to undertake a re-parameterization to the parameter $\theta^* \in \mathbb{R}$. We illustrate this for the Poisson distribution.

Example 11: In the Poisson distribution, $\mu > 0$. We could replace this parameter with $\theta^* = \ln \mu \in \mathbb{R}$.

```r
lnl_poisson4 <- function(thetastar, y) {
  mu <- exp(thetastar)
  lnl <- dpois(y, lambda = mu, log = TRUE)
  return(sum(lnl))
}
poisson_fit3 <- maxLik(lnl_poisson4,
                       method = "NR",
                       start = 1,
                       y = y_poisson)
summary(poisson_fit3)
```

```
--------------------------------------------
Maximum Likelihood estimation
Newton-Raphson maximisation, 4 iterations
Return code 8: successive function values within relative tolerance limit (reltol)
Log-Likelihood: -1748.701
1  free parameters
Estimates:
     Estimate Std. error t value Pr(> t)
[1,]  0.71784    0.02213   32.44  <2e-16 ***
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
--------------------------------------------
```

The Poisson distribution is now parameterized in terms of θ^*, which gets transformed to μ in line 2. For the log-likelihood, I have taken the shortcut of specifying `log = TRUE`. This has the same effect as using `log(dpois())`.

You will notice that the estimate that is being generated differs from that shown in Example 5. The reason is that we have estimated θ instead of μ. Consequently, the reported estimate is equal to the natural logarithm of the mean. We simply have to make this estimate the power of e to obtain μ.

For the standard errors, things are a bit more complicated. The standard error reported with the estimate is that for $\hat{\theta}$ and not for $\hat{\mu}$. To obtain the lattter, we could use the delta method (Doob 1935; Dorfman 1938). The `deltamethod` function in `msm` performs the computations.

Example 12: Since the output produces $\hat{\theta}^* = \ln \hat{\mu}$ and we are interested in $\hat{\mu}$, we perform the back-transformation $\hat{\mu} = \exp(\hat{\theta}^*)$.

```
library(msm)
log_mean_est <- coef(poisson_fit3)
log_mean_var <- vcov(poisson_fit3)
mean_est <- exp(log_mean_est)
mean_est
```

```
[1] 2.05
```

```
mean_var <- deltamethod(~exp(x1),
                        log_mean_est,
                        log_mean_var)
mean_var
```

```
[1] 0.04536758
```

Confidence Intervals

So far, we have focused on generating point estimates using maximum likelihood. We now consider two different procedures for generating interval estimates.

Wald Confidence Intervals

Wald (1943) proposed symmetric confidence intervals around the maximum likelihood estimator. These can be obtained using `confint`.

Example 13: The following syntax generates 95%-Wald confidence intervals for the parameters of the normal distribution.

```
confint(normal_fit1, level = 0.95)
```

```
     2.5 %    97.5 %
1 4.141567  5.820441
2 9.327456 10.596564
```

The first row shows the confidence interval for μ, whereas the second row does this for σ.

Profile Likelihood Confidence Intervals

Profile likelihood confidence intervals are potentially asymmetric (Pawitan 2001; Venzon and Moolgavkar 1988). There is no standard procedure of estimating these after `maxLik`, but the following function takes care of the computations.[4]

```r
profile_ci <- function(logLik_func, start, y, idx, level = 0.95, points = 500) {

  # Bind y to the log-likelihood function
  logLik_y <- function(pars) logLik_func(pars, y)

  # Fit the full model and get the maximum log-likelihood
  fit <- maxLik(logLik = logLik_y, start = start)
  max_logLik <- logLik(fit)

  # Chi-squared threshold for confidence level
  threshold <- qchisq(level, df = 1) / 2

  # Range around MLE for profiling
  est <- coef(fit)
  param_range <- seq(est[idx] - 3 * sqrt(vcov(fit)[idx, idx]),
                     est[idx] + 3 * sqrt(vcov(fit)[idx, idx]),
                     length.out = points)

  # Profile log-likelihood values
  prof_logLik <- numeric(length(param_range))
  for (i in seq_along(param_range)) {
    # Fix one parameter and optimize the rest
    fixed_val <- param_range[i]

    # Log-likelihood with fixed parameter
    prof_logLik_y <- function(params) {
      full_params <- est
      full_params[idx] <- fixed_val
      full_params[-idx] <- params
      logLik_y(full_params)
    }

    # Optimize with fixed parameter
    fit_prof <- maxLik(logLik = prof_logLik_y, start = est[-idx])
    prof_logLik[i] <- logLik(fit_prof)
  }
```

[4]Here, I set a grid of 500 values. For more precise computations, this number can be increased. However, the price is that the computations will take longer.

```
  # Interval limits where log-likelihood is within the threshold
  valid_idx <- which(prof_logLik >= (max_logLik - threshold))
  ci <- range(param_range[valid_idx])

  list(ci = ci, prof_logLik = prof_logLik, param_vals = param_range)
}
```

Example 14: We now apply the function to the normal distribution to derive the 95% profile likelihood confidence interval for the mean.

```
prof_ci_mu <- profile_ci(lnl_normal,
                         start = c(0,1),
                         idx = 1,
                         y = y_normal)
prof_ci_mu$ci
```

```
[1] 4.108113 5.853895
```

Simulations

In applied statistics, we are frequently interested in quantities that are functions of maximum likelihood estimators. One can think of predicted probabilities in a logit model or of the coefficient of variation. How do we estimate those quantities and how do we compute confidence intervals? We rely on three principles: (1) functions of maximum likelihood estimators are themselves maximum likelihood estimators; (2) assuming consistency for the function inputs, the output of that function is also a consistent estimator; and (3) asymptotically, the function output is normally distributed.[5]

These principles legitimize a simple way of simulating derived quantities. The recipe is as follows:

1. Make R draws from $N\left(\hat{\theta}, \hat{\mathbb{V}}[\hat{\theta}]\right)$, where R should be a large number.
2. For each draw, compute the quantity of interest as $f\left(\hat{\theta}^r\right)$, where $r = 1, \cdots, R$ and $f(.)$ yields the quantity of interest.
3. Compute the mean across the R estimators of $f\left(\hat{\theta}^r\right)$; this is the point estimate of the quantity of interest.
4. For confidence level α, compute the $\alpha/2$ and $1 - \alpha/2$ percentiles; these yield the confidence interval.

[5] The last two principles are consequences of Slutsky's theorem.

Example 15: The coefficient of variation is $\text{CV} = \sigma/\mu$. Using the normal distribution, we would simulate this quantity as follows:

```
library(MASS)
# Number of draws
R <- 1000
# Generate draws
theta <- coef(normal_fit1)
vce <- vcov(normal_fit1)
set.seed(10)
draws <- mvrnorm(R, mu = theta, Sigma = vce)
# For each draw, compute CV
cv <- draws[,2]/draws[,1]
# Compute the means and 2.5th and 97.5th percentiles
mean(cv)
```

```
[1] 2.015207
```

```
quantile(cv, probs = c(0.025, 0.975))
```

```
    2.5%    97.5%
1.692544 2.410385
```

The point estimate of the coefficient of variation is 2.015. The 95%-confidence interval runs from 1.693 to 2.41.

The Likelihood Ratio Test

As King (1998) argues, maximum likelihood unifies two important aspects of statistical inference: estimation and hypothesis testing. The likelihood ratio test provides a framework for the latter aspect. Imagine we have estimated two different models of the data generating process, M_U and M_R. $M_R \subset M_U$, i.e., there models are nested. This means that M_R comes about by restricting a subset of the parameters in M_U. Model comparison thus amounts to testing the constraints. This is done using

$$\text{LR} = -2\left[\ell_R - \ell_U\right] \overset{asy}{\sim} \chi^2_q$$

Here q is the number of restrictions that M_R imposes. The following function performs the LR-test after fitting two nested models.

```r
lr_test <- function(U,R) {
  lnl_u <- logLik(U)
  lnl_r <- logLik(R)
  q <- nParam(U, free = TRUE) - nParam(R, free = TRUE)
  LR <- 2 * (lnl_u - lnl_r)
  p <- pchisq(LR, q, lower.tail = FALSE)
  out <- matrix(c(lnl_u, lnl_r, LR, q, p),
                nrow = 1, ncol = 5)
  labels <- c("lnl.U", "lnl.R", "LR", "df", "Prob")
  colnames(out) <- labels
  return(out)
}
```

Example 16: In Example 8, we estimated the parameters of the normal probability density. In example 10, we imposed the constraint $H_0 : \sigma = 1$. We now perform the likelihood ratio test of the hypothesis

```r
lr_test(normal_fit1, c_normal_fit)
```

```
         lnl.U     lnl.R       LR df Prob
[1,] -1399.388 -24810.29 46821.81  1    0
```

Given the very small p-value, we reject H_0.

Advanced Models

Classical Regression

In matrix form, the classical linear regression model with normal errors is given by

$$y = X\beta + \varepsilon$$

Here, y is the response vector, X is a matrix of predictors, and β is a vector of regression coefficients. Further, $\varepsilon \sim N(0, \sigma^2 I)$. We seek to estimate the regression coefficients and the root mean squared error, σ. The log-likelihood function is given by

$$\ell = -n \ln \sigma - \frac{1}{2\sigma^2} (y - X\beta)^\top (y - X\beta)$$

Since $\varepsilon = y - X\beta$, we can also write the log-likelihood as

$$\ell = -n \ln \sigma - \frac{1}{2\sigma^2} \varepsilon^\top \varepsilon$$

One way to program this log-likelihood is using the following syntax

```
lnl_clm <- function(pars, y, X) {
  n <- nrow(X)
  K <- ncol(X)
  beta <- pars[1:K]
  sigma <- pars[K + 1]
  e <- y - X %*% beta
  lnl <- -n * log(sigma) - (1/(2 * sigma^2)) * t(e) %*% e
  return(lnl)
}
```

Here, `%*%` is the operator for matrix multiplication, whereas `t(e)` takes the transpose of the errors contained in ε. Notice that instead of specifying one data object, I have now generated objects for the response variable (`y`) and the predictors (`X`)

Example 17:

We illustrate the linear regression model with the Boston Housing data. We predict median housing values in Boston precincts using crime rate and taxes as the predictors. Note that we add a column of 1s at the beginning of `X`.

```
library(mlbench)
data(BostonHousing2)
y <- as.matrix(BostonHousing2$cmedv,
               ncol = 1, byrow = TRUE)
X <- cbind(1,
           BostonHousing2$crim,
           BostonHousing2$tax)
reg_fit <- maxLik(lnl_clm,
                  method = "NR",
                  start = c(rep(0, 3), 10),
                  y = y,
                  X = X)
summary(reg_fit)
```

```
--------------------------------------------
Maximum Likelihood estimation
```

```
Newton-Raphson maximisation, 11 iterations
Return code 8: successive function values within relative tolerance limit (reltol)
Log-Likelihood: -1304.127
4  free parameters
Estimates:
      Estimate Std. error t value  Pr(> t)
[1,] 31.448981   1.425499  22.062  < 2e-16 ***
[2,] -0.185100   0.055707  -3.323 0.000891 ***
[3,] -0.020211   0.003494  -5.784 7.28e-09 ***
[4,]  7.983187   0.248886  32.076  < 2e-16 ***
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
--------------------------------------------
```

Heteroskedastic Regression

Harvey (1976) proposed the heteroskedastic regression model where

$$\sigma_i^2 = \exp\left(z_i^\top \gamma\right)$$

Further, $y_i = x_i^\top \beta$ and $y_i \sim N(\mu_i, \sigma_i)$. With these assumptions in place, the log-likelihood kernel is

$$\ell = -.5 \sum_{i=1}^{n} z_i^\top \gamma - .5 \sum_{i=1}^{n} \frac{\left(y_i - x_i^\top \beta\right)^2}{\exp\left(z_i^\top \gamma\right)}$$

We assume that x and z contain P and Q variables, respectively. This includes the constants. We program the log-likelihood as:

```
lnl_harvey <- function(pars, y, x, z) {
  P <- ncol(x)
  Q <- ncol(z)
  beta <- pars[1:P]
  gamma <- pars[(P + 1):(P + Q)]
  lp1 <- x %*% beta
  lp2 <- z %*% gamma
  lnl <- -.5 * lp2 -.5 * ((y - lp1)^2)/exp(lp2)
  return(sum(lnl))
}
```

Line 2 defines the number of predictors in x, whereas line 3 does the same for z. Lines 4 and 5 partition the parameter vector into β and γ. Line 6 defines the linear predictor $x_i^\top \beta$, while line 7 defines the linear predictor $z_i^\top \gamma$. Line 8 defines ℓ_i, the log-likelihood contribution for a single observation., whereas line 9 returns the value of this contribution.

Example 18: We theorize that the expected median housing value depends on taxes and crime. Simultaneously, we hypothesize that the spread in median housing values depends on the crime rate. The higher the crime rate, the less dispersion in housing values is expected.

```
y <- BostonHousing2$cmedv
x <- cbind(1, BostonHousing2$crim, BostonHousing2$tax)
z <- cbind(1, BostonHousing2$crim)
harvey_fit <- maxLik(lnl_harvey,
                     method = "NR",
                     start = rep(0,5),
                     y = y, x = x, z = z)
summary(harvey_fit)
```

```
--------------------------------------------
Maximum Likelihood estimation
Newton-Raphson maximisation, 31 iterations
Return code 8: successive function values within relative tolerance limit (reltol)
Log-Likelihood: -1302.462
5  free parameters
Estimates:
      Estimate Std. error t value  Pr(> t)
[1,] 31.837024   0.679904  46.826  < 2e-16 ***
[2,] -0.143010   0.034366  -4.161 3.16e-05 ***
[3,] -0.021596   0.001810 -11.931  < 2e-16 ***
[4,]  4.211495   0.070149  60.036  < 2e-16 ***
[5,] -0.017711   0.008832  -2.005   0.0449 *
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
--------------------------------------------
```

The results are consistent with our expectations. A higher crime rate tends to reduce median housing values and also makes them more predictable (lower variance).

Alternative Distributional Assumptions

The normal distribution is quite restrictive in that it does not allow for light- or heavy-tailed distributions. The exponential power distribution (a.k.a. Type-II generalized normal distribution) allows for more flexibility vis-à-vis kurtosis. Its probability density function is

$$p(y) = \frac{\lambda}{2\,\sigma\,\Gamma(1/\lambda)} \exp\left\{ -\left(\frac{|y-\mu|}{\sigma} \right)^{\lambda} \right\}$$

Here λ is a shape parameter such that (1) $\lambda < 2$ results in heavy-tailed, (2) $\lambda > 2$ in light-tailed, and (3) $\lambda = 2$ in normal distributions.

The log-likelihood contribution of the ith unit is

$$\ell_i = \ln\lambda - \ln\sigma - \ln\Gamma\left(\lambda^{-1}\right) - \left(\frac{|y_i - \mu_i|}{\sigma} \right)^{\lambda}$$

where $\mu_i = x_i^{\top}\beta$. We can program this as follows.

```
lnl_expow <- function(pars, y, x) {
  n <- length(y)
  P <- ncol(x)
  beta <- pars[1:P]
  sigma <- pars[P+1]
  lambda <- pars[P+2]
  if (sigma <= 0 || lambda <= 0) {
    return(-Inf)
  }
  lp <- x %*% beta
  gamma_term <- gamma(1/lambda)
  lnl <- n * log(lambda) - n * log(2) - n * log(sigma) -
    n * log(gamma_term) - sum((abs(y - lp) / sigma)^lambda)
  return(lnl)
}
```

We now apply this to Boston Housing data.

Example 19: Using the data objects generated in Example 18, we get the following results.

```
expow_fit <- maxLik(lnl_expow,
                    method = "NR",
                    start = c(rep(10,3), 10, 2),
                    y = y, x = x)
summary(expow_fit)
```

```
--------------------------------------------
Maximum Likelihood estimation
Newton-Raphson maximisation, 11 iterations
```

```
Return code 3: Last step could not find a value above the current.
Boundary of parameter space?
Consider switching to a more robust optimisation method temporarily.
Log-Likelihood: -3408.407
5  free parameters
Estimates:
       Estimate Std. error  t value Pr(> t)
[1,]  4.598e+01  6.566e-01   70.026  <2e-16 ***
[2,] -9.379e+01  5.565e-01 -168.536  <2e-16 ***
[3,]  1.162e-02  9.505e-04   12.223  <2e-16 ***
[4,]  9.048e-02  6.359e-02    1.423   0.155
[5,]  2.225e-01  1.322e-02   16.827  <2e-16 ***
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
-----------------------------------------
```

Logit and Probit Models

Consider the binary outcome $y_i \in \{0, 1\}$. The 1s occur with probability π_i and the 0s with probability $1 - \pi_i$. Further, $\pi_i = F(x_i^\top \beta)$, where $F(.)$ is assumed to be the standard logistic or the standard normal distribution function.

The log-likelihood for a single unit derives from the Bernoulli distribution:

$$P(y_i) = \pi_i^{y_i} \cdot (1 - \pi_i)^{1-y_i}$$

By taking the natural logarithm, the log-likelihood contribution of the ith unit is:

$$\ell_i = y_i \cdot \ln \pi_i + (1 - y_i) \cdot \ln (1 - \pi_i)$$

We note that for the standard logistic and normal distributions, $1 - \pi_i = F(-x_i^\top \beta)$. This allows us to simplify ℓ_i. Let $q_i = 2y_i - 1$ (Greene 2020). Then

$$\ell_i = \ln F(q_i x_i^\top \beta)$$

We now program the logistic log-likelihood as follows.

```r
lnl_logit <- function(pars, y, x) {
  P <- ncol(x)
  beta <- pars[1:P]
  lp <- x %*% beta
  q <- 2 * y - 1
```

```
  lnl <- sum(log(plogis(q * lp)))
  return(lnl)
}
```

Line 2 computes the number of predictors as P. Line 3 declares the vector β as the 1st through Pth elements of `pars`. Line 4 defines the linear predictor, whereas line 5 defines q_i. The log-likelihood is defined in line 6. We could simply replace `plogis` with `pnorm` to change the code to the probit model. Finally, line 7 returns the log-likelihood.

Example 20: We apply the code to Ronald Fisher's famous iris data. Here, we try to predict whether an iris is of the species versicolor. We prepare the data as follows.

```
library(tidyverse)
data(iris)
iris <- iris |>
  mutate(sl = scale(Sepal.Length),
         sw = scale(Sepal.Width),
         pl = scale(Petal.Length),
         pw = scale(Petal.Width),
         y = ifelse(Species == "versicolor", 1, 0))
y <- iris$y
x <-  cbind(1, iris$sl, iris$sw, iris$pl,
            iris$pw)
```

Note that we standardize the predictor variables to facilitate convergence.

Instead of making up starting values, we adopt the estimates from the linear probability model.

```
lpm_fit <- lm(y ~ x - 1)
st_val <- coef(lpm_fit)
```

We drop the intercept because `x` already includes one. The logit estimation yields the following results.

```
logit_fit <- maxLik(lnl_logit,
                    method = "NR",
                    start = st_val,
                    y = y, x = x)
summary(logit_fit)
```

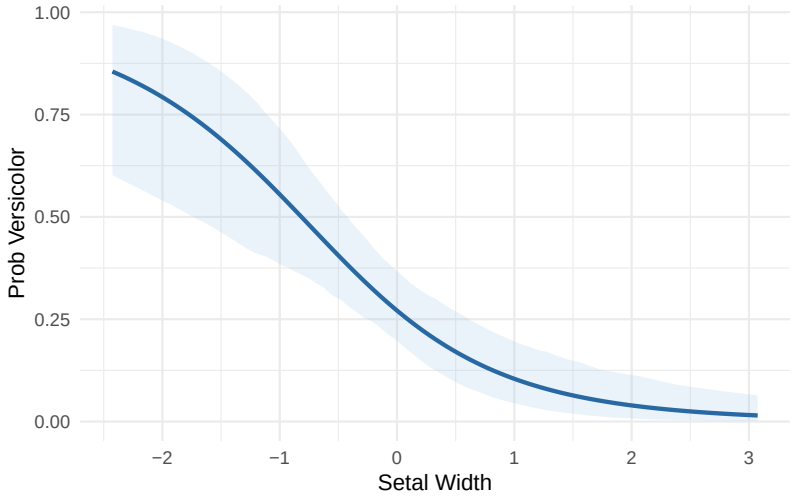
```
--------------------------------------------
Maximum Likelihood estimation
Newton-Raphson maximisation, 5 iterations
Return code 8: successive function values within relative tolerance limit (reltol)
Log-Likelihood: -72.53484
5  free parameters
Estimates:
   Estimate Std. error t value  Pr(> t)
x1  -1.0007     0.2260  -4.428 9.52e-06 ***
x2  -0.2032     0.5239  -0.388 0.698192
x3  -1.2189     0.3343  -3.647 0.000266 ***
x4   2.3190     1.1395   2.035 0.041842 *
x5  -2.1177     0.8548  -2.477 0.013232 *
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
--------------------------------------------
```

We use the logistic results to show how one can obtain predicted probabilities and their confidence intervals through simulation.

Example 21: We are interested in the effect of setal width on the predicted probability that the iris is of the type versicolor. We hold the remaining predictors at their mean values in the sample.

```r
sw <- seq(min(iris$sw), max(iris$sw), .05)
sl <- mean(iris$sl)
pl <- mean(iris$pl)
pw <- mean(iris$pw)
R <- 1000
beta <- coef(logit_fit)
vce <- vcov(logit_fit)
set.seed(25)
beta_draws <- mvrnorm(R, beta, vce)
x <- cbind(1, sl, sw, pl, pw)
probs <- plogis(beta_draws %*% t(x))
mean_preds <- colMeans(probs)
lower_preds <- apply(probs, 2, quantile, probs = 0.025)
upper_preds <- apply(probs, 2, quantile, probs = 0.975)
plot_df <- data.frame(
  sw = sw,
  mean = mean_preds,
  lower = lower_preds,
  upper = upper_preds
)
```

```r
ggplot(plot_df, aes(x = sw)) +
  geom_line(aes(y = mean), color = "#0b5394", linewidth = 1) +
  geom_ribbon(aes(ymin = lower, ymax = upper), fill = "#9fc5e8", alpha = 0.2) +
  labs(
    x = "Setal Width",
    y = "Prob Versicolor"
  ) +
  theme_minimal()
```



Poisson Regression

The Poisson regression model is perhaps the most basic of the count models. It models counts y as a function of predictors using the Poisson distribution. Specifically,

$$P(y_i) = \frac{\mu_i^{y_i} \exp(-\mu_i)}{y_i!}$$

where $\mu_i > 0$ is the conditional mean (and variance) of the count. We use a logarithmic link function such that

$$\ln \mu_i = x_i^\top \beta$$

The kernel of the log-likelihood function is given by

$$\ell = \sum_{i=1}^{n} y_i \ln \mu_i - \sum_{i=1}^{n} \mu_i = \sum_{i=1}^{n} y_i x_i^\top \beta - \sum_{i=1}^{n} \exp\left(x_i^\top \beta\right)$$

We program this as follows.

```
lnl_poisson <- function(pars, y, x) {
  P <- ncol(x)
  beta <- pars[1:P]
  lp <- x %*% beta
  lnl <- sum(y * lp) - sum(exp(lp))
  return(lnl)
}
```

Example 22: We apply the code to the `warpbreaks` data set, which counts the number of breaks in a yarn weaving process. We have two categorical predictors: (1) `wool`, the type of wool (2 levels) and (2) `tension`, the tension during the weaving process (3 levels). We use the `dummy` package to turn the factors into dummies.

```
library(fastDummies)
data("warpbreaks")
work <- dummy_cols(warpbreaks,
                   select_columns = c("wool", "tension"),
                   remove_first_dummy = TRUE,
                   remove_selected_columns = TRUE)
y <- work$breaks
x <- cbind(1, work$wool_B, work$tension_M, work$tension_H)
```

We now estimate the Poisson regression model.

```
poisson_fit <- maxLik(lnl_poisson,
                      method = "NR",
                      start = rep(0, 4),
                      y = y, x = x)
summary(poisson_fit)
```

```
--------------------------------------------
Maximum Likelihood estimation
Newton-Raphson maximisation, 6 iterations
Return code 8: successive function values within relative tolerance limit (reltol)
Log-Likelihood: 3596.462
4  free parameters
Estimates:
     Estimate Std. error t value  Pr(> t)
[1,]  3.69196    0.04536  81.388  < 2e-16 ***
[2,] -0.20599    0.05154  -3.997 6.42e-05 ***
```

```
[3,] -0.32132    0.06025  -5.334 9.63e-08 ***
[4,] -0.51849    0.06411  -8.088 6.07e-16 ***
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
--------------------------------------------
```

References

Akaike, Hirotugu. 1973. "Information Theory and Extension of the Maximum Likelihood Principle." In *2nd International Symposium on Information Theory, Tsahkadsor, Armenia, USSR, September 2-8, 1971*, edited by Boris N. Petrov and Frigyes Csáki, 267–81. Budapest: Akadémiai Kiadó.

Berndt, Ernst R., Bronwyn Hall, Robert Hall, and Jerry Hausman. 1974. "Estimation and Inference in Nonlinear Structural Models." *Annals of Economic and Social Measurement* 3 (4): 653–65.

Broyden, C. G. 1970. "The Convergence of a Class of Double-Rank Minimization Algorithms 1. General Considerations." *IMA Journal of Applied Mathematics* 6 (1): 76–90. https://doi.org/10.1093/imamat/6.1.76.

Doob, J. L. 1935. "The Limiting Distributions of Certain Statistics." *The Annals of Mathematical Statistics* 6 (3): 160–69. https://doi.org/10.1214/aoms/1177732594.

Dorfman, Robert. 1938. "A Note on the ð-Method for Finding Variance Formulae." *The Biometric Bulletin* 1: 129–37.

Fletcher, R. 1970. "A New Approach to Variable Metric Algorithms." *The Computer Journal* 13 (3): 317–22. https://doi.org/10.1093/comjnl/13.3.317.

Goldfarb, Donald. 1970. "A Family of Variable-Metric Methods Derived by Variational Means." *Mathematics of Computation* 24 (109): 23–26. https://doi.org/10.1090/S0025-5718-1970-0258249-6.

Greene, William H. 2020. *Econometric Analysis*. Eighth edition, global edition. Harlow London New York Boston San Francisco Toronto Sydney Dubai Singapore Hong Kong Tokyo Seoul Taipei New Delhi Cape Town São Paulo Mexico City Madrid Amsterdam Munich Paris Milan: Pearson.

Harvey, A. C. 1976. "Estimating Regression Models with Multiplicative Heteroscedasticity." *Econometrica* 44 (3): 461–65. https://doi.org/10.2307/1913974.

Henningsen, Arne, and Ott Toomet. 2011. "maxLik: A Package for Maximum Likelihood Estimation in R." *Computational Statistics* 26 (3): 443–58. https://doi.org/10.1007/s00180-010-0217-1.

King, Gary. 1998. *Unifying Political Methodology: The Likelihood Theory of Statistical Inference*. Ann Arbor, MI: University of Michigan Press. https://doi.org/10.3998/mpub.23784.

Kirkpatrick, S., C. D. Gelatt, and M. P. Vecchi. 1983. "Optimization by Simulated Annealing." *Science* 220 (4598): 671–80. https://doi.org/10.1126/

science.220.4598.671.

Nelder, J. A., and R. Mead. 1965. "A Simplex Method for Function Minimization." *The Computer Journal* 7 (4): 308–13. https://doi.org/10.1093/comjnl/7.4.308.

Pawitan, Yudi. 2001. *In All Likelihood: Statistical Modelling and Inference Using Likelihood*. Oxford : New York: Clarendon Press ; Oxford University Press.

Shanno, D. F. 1970. "Conditioning of Quasi-Newton Methods for Function Minimization." *Mathematics of Computation* 24 (111): 647–56. https://doi.org/10.1090/S0025-5718-1970-0274029-X.

Shewchuk, Jonathan Richard. 1994. "An Introduction to the Conjugate Gradient Method Without the Agonizing Pain." Pittsburgh, PA.

Thisted, Ronald A. 1988. *Elements of Statistical Computing: Numerical Computation.* New York: Chapman & Hall. https://www.taylorfrancis.com/books/e/9781351452755.

Venzon, D. J., and S. H. Moolgavkar. 1988. "A Method for Computing Profile-Likelihood-Based Confidence Intervals." *Applied Statistics* 37 (1): 87. https://doi.org/10.2307/2347496.

Wald, Abraham. 1943. "Tests of Statistical Hypotheses Concerning Several Parameters When the Number of Observations Is Large." *Transactions of the American Mathematical Society* 54 (3): 426–82. https://doi.org/10.1090/S0002-9947-1943-0012401-3.